



Problem Statement

Problem

You are given a grid of dimensions $N \times M$. Each cell contains one of the following values:

0 → Empty cell 1 → Fresh orange 2 → Rotten orange

A rotten orange can rot an adjacent fresh orange in 1 unit of time.

A rotten orange at position (i, j) can rot fresh oranges at:

$(i-1, j) \rightarrow (i+1, j) \rightarrow (i, j-1) \rightarrow (i, j+1) \rightarrow$ Right

Find the minimum time required to rot all fresh oranges.

If it is impossible to rot all fresh oranges, print -1.

If there are no fresh oranges initially, the required time is 0.

Input Format

The first line contains an integer T , the number of test cases.

For each test case: The first line contains two integers N and M , representing the number of rows and columns. The next N lines contain M space-separated integers representing the grid.

Constraints

$1 \leq T \leq 100$ $1 \leq N, M \leq 500$ $\text{grid}[i][j] \in \{0, 1, 2\}$

Output Format

Print a single integer representing the minimum time required to rot all fresh oranges.

Print -1 if some fresh oranges can never become rotten

Sample Input 0

```
3 3
2 1 1
1 1 0
0 1 1
```

Sample Output 0

```
4
```

Sample Input 1

```
3 3
2 1 1
0 1 1
1 0 1
```

Sample Output 1

```
-1
```

Change Theme

Language

Java 15



```
1 import java.io.*;
2 import java.util.*;
3
4 public class Solution {
5
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8
9         int n = sc.nextInt();
10        int m = sc.nextInt();
11
12        int[][] grid = new int[n][m];
13
14        Queue<int[]> q = new LinkedList<>();
15        int fresh = 0;
16
17        for (int i = 0; i < n; i++) {
18            for (int j = 0; j < m; j++) {
19                grid[i][j] = sc.nextInt();
20                if (grid[i][j] == 2) q.add(new int[] {i, j});
21                if (grid[i][j] == 1) fresh++;
22            }
23        }
24
25        if (fresh == 0) {
26            System.out.println(0);
27            return;
28        }
29
30        int time = 0;
31        while (!q.isEmpty()) {
32            int size = q.size();
33            for (int i = 0; i < size; i++) {
34                int[] cell = q.poll();
35                int r = cell[0], c = cell[1];
36                int[] dr = {-1, 1, 0, 0}, dc = {0, 0, -1, 1};
37                for (int d = 0; d < 4; d++) {
38                    int nr = r + dr[d], nc = c + dc[d];
39                    if (nr < 0 || nr > n-1 || nc < 0 || nc > m-1 || grid[nr][nc] != 1) continue;
40                    grid[nr][nc] = 2;
41                    q.add(new int[] {nr, nc});
42                }
43            }
44            time++;
45        }
46
47        if (fresh > 0) System.out.println(-1);
48        else System.out.println(time-1);
49    }
50 }
```

Line: 81 Col: 1

Upload Code as File

Run Code

Submit Code

☐ Test against custom input

Test case 0

Test case 1

Compiler Message

Success